Architecture technique et plan d’initialisation

Simulation multi-agents du lancement d’une crème solaire (marque fictive) : 120 agents, modèles open weight en local, Kafka, RAG et visualisation Godot.

Sujet	Sujet 04 ÉPHÉMÈRE : viralité d’une tendance	**Version**	Brouillon v1 · 14 octobre 2026
Machine cible	HP Omen 16 : RTX 5060 (8 Go de VRAM, à confirmer), Ryzen 9 8945HX, 32 Go de RAM, SSD de 1 To	**Statut**	À valider par le binôme avant de coder

Principe directeur. Cent vingt agents ne peuvent pas tous appeler un LLM à chaque pas sur un portable à 8 Go de VRAM. L’architecture repose sur trois leviers : **(1)** trois paliers de modèles Qwen3 de la même famille (1,7 B, 4 B, 8 B) complétés par des agents à règles ; **(2)** des appels LLM **déclenchés par événement** et non à chaque pas ; **(3)** un **ordonnancement par palier** pour limiter les chargements de modèles en mémoire graphique.

1. Décisions clés en un coup d’œil

Sujet	Choix retenu	Pourquoi
Inférence LLM	Ollama installé sur l’hôte, modèles **Qwen3 1,7 b / 4 b / 8 b** + modèle d’embeddings	Gratuit, local, une seule famille : seule la taille varie entre profils (A5). Les trois tailles passent dans 8 Go, pas ensemble.
Bus d’événements	Apache Kafka, un broker, mode KRaft, sous Docker	Imposé (C1). Ordre, rejeu, traçabilité, découplage entre simulation, Godot et indicateurs.
État courant	Redis	Lectures et écritures fréquentes, latence de quelques millisecondes, données éphémères (F3).
Historique des runs	Fichiers Parquet interrogés avec DuckDB	Analytique reproductible (F5), fichier portable, rejeu (E3), aucun serveur à gérer.
Base de connaissances	ChromaDB local + embeddings Ollama	Recherche par similarité avec filtres (agent, date simulée) pour le RAG (F1, F2, C6).
Orchestration	Orchestrateur Python asyncio écrit par le binôme	Contrôle total de l’horloge simulée, des deux modes (C3, C4) et du budget d’appels. Un framework multi-agents le masquerait.
Visualisation	Godot 4 relié par un pont WebSocket (FastAPI)	Godot ne consomme pas Kafka directement ; le pont agrège et sert aussi l’inspection (D2) et l’injection (D3).
APIs gratuites	Facultatives : renfort, jamais le chemin principal	Quotas faibles et changeants (voir section 12). Le rejeu (E3) est le filet de sécurité en démonstration.

2. Le monde et les exigences qui structurent l’architecture

Monde retenu. Un été simulé (proposition : 90 jours, un pas par jour) autour du lancement d’une crème solaire identique pour tous, par une marque fictive dont le nom reste à définir (il sera un simple paramètre {BRAND_NAME} pour que les agents ne s’appuient jamais sur une marque réelle). Sept zones : pharmacie, plage, piscine, réseaux sociaux, centre commercial, terrain de sport en plein air, domicile. Le sens partagé (B1) se mesure sur des catégories fermées, proposées pour l’instant : *efficace*, *tendance*, *banal*, *dépassé*.

Exigences de l’énoncé	Ce qu’elles imposent à l’architecture
A4, A5, A6	Au moins deux tailles de modèles attribuées par rôle et expérience (jamais par catégorie sociale), agents à règles justifiés, sorties invalides tolérées et comptées.
C1, C2, F4	Événements publiés sur Kafka, nombre d’agents fixé au lancement sans modifier le code, schémas documentés et versionnés, événements non conformes isolés.

Exigences de l'énoncé	Ce qu'elles imposent à l'architecture
C3, C4, C5, C6	Horloge simulée explicite, distinction entre moment simulé et moment de traitement, modes synchrone et asynchrone comparés, aucune information « du futur ».
F1, F2, F3	RAG réel, qualité de récupération démontrée, choix de stockage justifié par le volume, la fréquence, la latence et la durée de conservation.
D1, D2, D3	Affichage en direct dans Godot, inspection d'un agent (mémoire, justification), injection d'un événement en direct.
E1 à E5	Baseline à règles, au moins 5 runs sur 2 configurations, rejeu sans appel LLM, mesure du coût de chaque run, gain de la répartition des modèles par rapport au « tout grand modèle ».
B1, B2, B3	Sens partagé défini en termes observables, indicateur calculé pendant le run, démonstration qu'il distingue un run avec émergence d'un run sans.

3. Ce que la machine impose

La contrainte dominante est la **mémoire de la carte graphique**, pas la RAM ni le processeur. Les tailles ci-dessous sont des ordres de grandeur en quantification Q4, à vérifier avec `ollama ps` et `nvidia-smi`.

Poste de VRAM (8 Go au total)	Estimation	Remarque
qwen3:1.7b (palier petit)	≈ 1,4 Go	Très rapide, reste chargé en permanence.
qwen3:4b (palier moyen)	≈ 2,5 Go	Reste chargé avec le petit modèle.
qwen3:8b (palier grand)	≈ 5,2 Go	Ne tient pas avec les deux autres : chargé par phase.
Modèle d'embeddings (RAG)	≈ 0,3 Go	Petit modèle dédié, garde sa place.
Cache de contexte (KV)	≈ 0,5 à 1 Go	Croît avec le nombre de requêtes parallèles.
Windows + Godot	≈ 1 à 1,5 Go	À déduire du budget disponible.
Phase A : petit + moyen + embeddings	≈ 6,2 Go	Tient avec marge.
Phase B : grand + embeddings	≈ 7,2 Go	Tient de justesse : parallélisme limité.

RAM (32 Go). Estimation : Windows 4 à 6 Go, Docker/WSL2 (Kafka avec un tas limité à 1 Go, Redis) 4 à 8 Go, Ollama hors VRAM 2 à 3 Go, orchestrateur Python 1 à 2 Go, ChromaDB 0,5 à 1 Go, Godot 1 à 2 Go, soit environ 16 à 22 Go : confortable, à condition de limiter WSL2. **Processeur (16 cœurs).** Largement suffisant pour Kafka, les 36 agents à règles et les indicateurs. **Portable.** Prévoir secteur, profil de performance maximal et surveillance thermique : un bridage du GPU fausserait les mesures de temps (E4).

4. Architecture d'ensemble

Les agents ne s'appellent jamais directement : tout passe par des événements Kafka, ce qui permet l'enregistrement complet d'un run, son rejeu et le calcul des indicateurs sans toucher à la simulation. Le tableau détaille les composants ; le schéma qui suit montre leurs liaisons.

Composant	Rôle	Exigences servies
Orchestrateur	Fait avancer l'horloge, déclenche les agents, regroupe les appels LLM par palier, applique timeouts et replis.	C2, C3, C4, A4
Agents LLM / agents à règles	Décident selon leur rôle ; les agents à règles portent la mécanique (stocks, promotions, météo, loyauté).	A1 à A6, E5
Kafka	Journal ordonné de tous les événements, avec schémas versionnés et file des événements non conformes (dlq).	C1, F4, F6
Redis	État courant du monde (stocks, météo, état par zone), lu par agents et pont.	F3, D1
ChromaDB	Deux collections : corpus fictif du monde et mémoires vécues des agents.	F1, F2, A1
DuckDB + Parquet	Historique de tous les runs, requêtes comparant les scénarios, journaux de rejeu, coûts.	F5, E3, E4

Composant	Rôle	Exigences servies
Service d’indicateurs	Calcule en direct l’entropie du sens, l’adoption et les écarts entre groupes ; chaque valeur renvoie aux événements bruts.	B2, B3, F6
Pont FastAPI + Godot	Affiche la scène et les indicateurs, permet d’inspecter un agent et d’injecter un événement.	D1, D2, D3

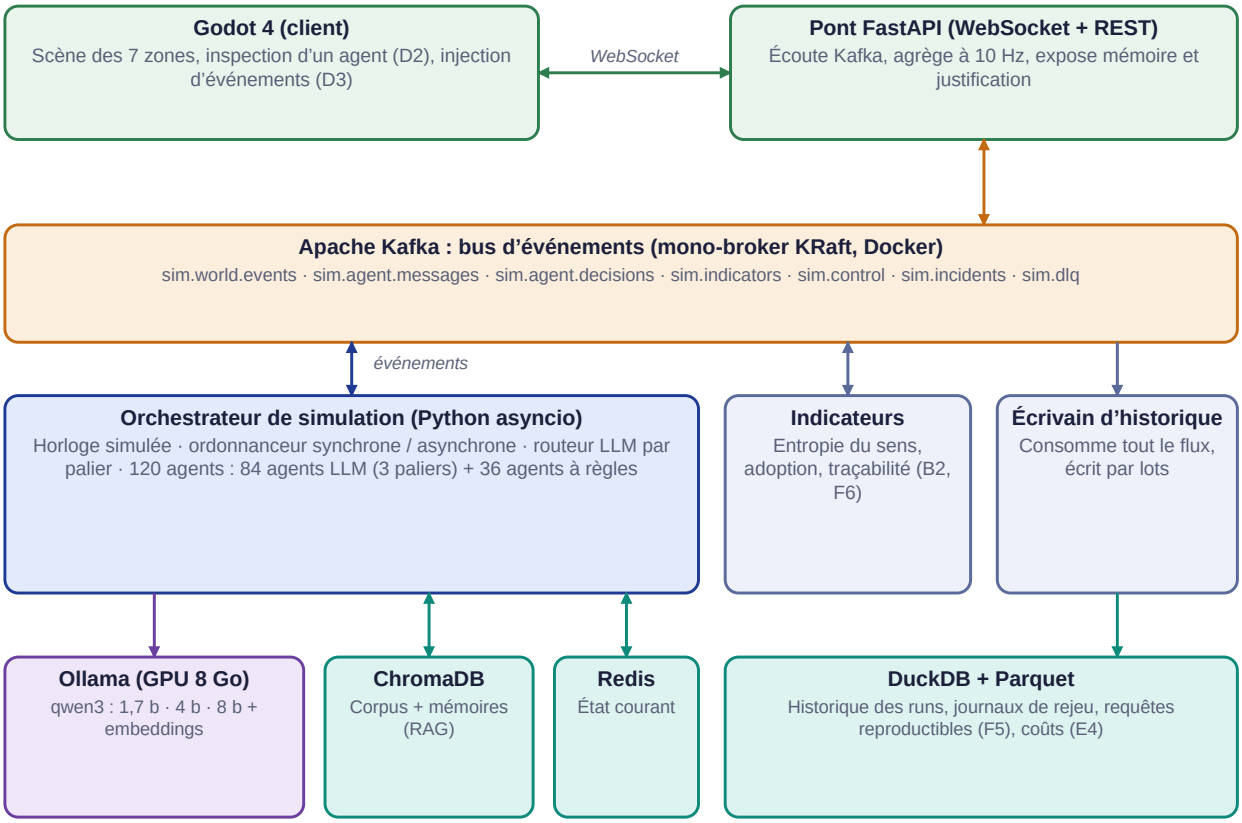


Schéma d’ensemble. Les flèches indiquent le sens principal des données.

5. Les acteurs et leurs modèles

Règle d’attribution. Le palier dépend du **rôle et de l’expérience** du profil, jamais d’une catégorie sociale (âge, sexe, revenu, origine), comme l’exige A5. Les seuils d’adoption hétérogènes (Granovetter) sont des **paramètres individuels** tirés dans une distribution, pas des attributs sociaux. Les 120 agents ci-dessous sont un exemple de dimensionnement : le nombre se règle au lancement (C2) et les proportions restent valables.

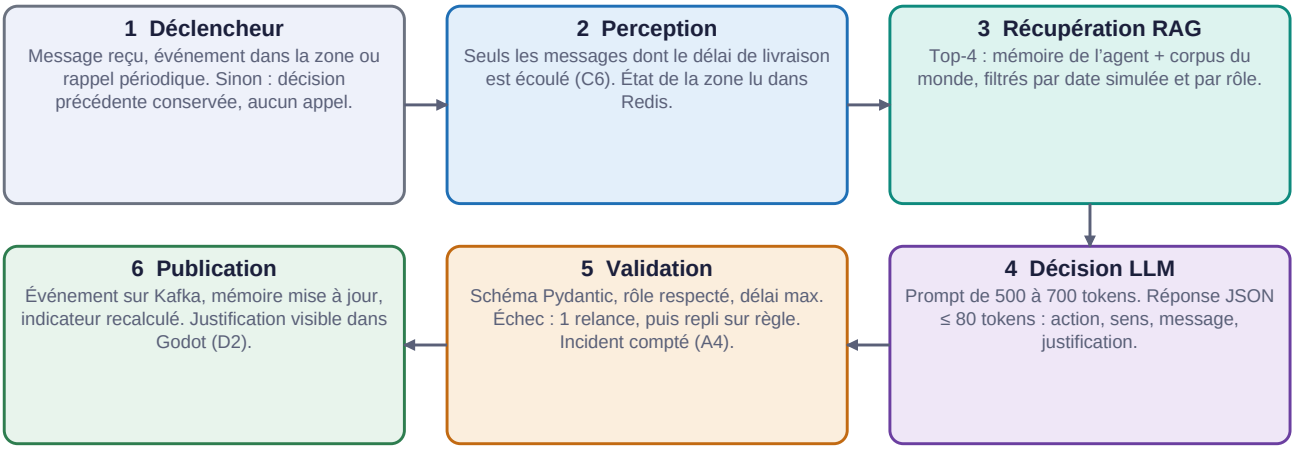
Rôle	Nb	Palier	Modèle	Déclencheur d’appel LLM	Appels / jour
Pharmaciens, conseillers	6	Grand	qwen3:8b	Question reçue, communiqué de la marque, rupture de stock (≈ 50 % des jours)	≈ 3,0
Créateurs de contenu beauté	6	Moyen	qwen3:4b	Tendance perçue, publication planifiée (≈ 40 %)	≈ 2,4
Journalistes beauté	3	Moyen	qwen3:4b	Communiqué ou polémique (≈ 20 %)	≈ 0,6
Responsable de la marque	1	Moyen	qwen3:4b	Bilan hebdomadaire des indicateurs (≈ 14 %)	≈ 0,1
Habitués du soin solaire (expérimentés)	16	Moyen	qwen3:4b	Message d’un proche, avis lu (≈ 30 %)	≈ 4,8
Baigneurs occasionnels	36	Petit	qwen3:1.7b	Exposition à un porteur ou à un message (≈ 25 %)	≈ 9,0
Sceptiques pragmatiques	16	Petit	qwen3:1.7b	Pression de l’entourage (≈ 25 %)	≈ 4,0
Fidèles à une autre marque	22	Règles	aucun	Loyauté fixe, seuil de changement paramétré	0
Vendeurs et caissiers	12	Règles	aucun	Stock, promotions, rayonnage	0

Rôle	Nb	Palier	Modèle	Déclencheur d'appel LLM	Appels / jour
Marque concurrente	1	Règles	aucun	Calendrier scripté de promotions et de lancements	0
Météo et saison	1	Règles	aucun	Horloge : indice UV, chaleur, affluence	0
Total	**120**			84 agents LLM (70 %) + 36 agents à règles (30 %)	**≈ 24**

Pourquoi ce découpage. Les LLM servent là où le sens se **négocie par la conversation** (conseil, prescription, avis). Les règles portent la mécanique (stocks, promotions, météo, loyauté), qu'un LLM résoudrait plus lentement sans rien apporter. Cette répartition est aussi le levier de sobriété demandé en E5. Le palier *grand* est réservé aux 6 pharmaciens, dont la crédibilité pèse le plus sur les autres.

> **Une seule famille de modèles.** En gardant Qwen3 aux trois tailles, la différence de comportement entre profils ne vient que de la capacité du modèle, ce qui est l'hypothèse de travail de A5. Mélanger des familles introduirait un second facteur de variation impossible à isoler, et affaiblirait la réponse à la question du jury sur la ressemblance des réponses d'un même modèle.

6. Boucle de décision d'un agent LLM



Sortie structurée. L'agent répond dans un schéma JSON imposé (paramètre `format` d'Ollama, pris en charge par les versions récentes) : `action`, `sens` parmi les catégories fermées, `message` facultatif, `justification` courte. Les catégories fermées réduisent les erreurs des petits modèles, la longueur des réponses et la tentation de réciter des connaissances réelles.

Réduire la latence.

- Mode « réflexion » désactivé pour les modèles qui en disposent (paramètre `think` ou instruction dédiée, selon la version d'Ollama).
- Réponses courtes : nombre de tokens générés plafonné, décision plus une phrase.
- Appel seulement sur déclencheur : un agent dont la situation n'a pas changé conserve sa décision précédente.
- Modèles gardés en mémoire (`OLLAMA_KEEP_ALIVE`) et requêtes regroupées par modèle pour éviter les rechargements.

7. Temps, ordonnancement et dimensionnement

Horloge simulée (C3). Chaque événement porte deux dates distinctes : le moment simulé où il se produit et le moment simulé où il est traité, plus l'horodatage réel d'émission. Un message envoyé le jour *t* n'est lisible qu'à partir de *t* + délai de livraison : c'est ce qui garantit la cohérence temporelle (C6) et la reproductibilité.

	Mode synchrone	Mode asynchrone
Principe	Barrière à chaque pas : tous les agents lisent l'état du pas précédent, puis publient.	Chaque agent agit à son rythme, latences réelles des LLM comprises.
Ordonnancement	Par palier : appels du palier grand, puis moyen, puis petit, puis règles. Un seul chargement de modèle par pas.	Appels mélangés : risque de rechargements fréquents dans 8 Go de VRAM.
Hypothèse à tester (C5)	Plus reproductible, plus rapide par appel grâce au regroupement.	Les profils rapides (petits modèles) influencent davantage le phénomène avant les profils lents.

Dimensionnement estimé pour un run (90 jours, 120 agents). Hypothèses de latence par appel (prompt d'environ 600 tokens, réponse d'environ 70 tokens) à remplacer par les mesures du benchmark de l'étape 8 ; ce ne sont pas des valeurs mesurées sur votre machine.

Palier	Appels / jour	Appels / run	Latence supposée	Durée séquentielle
Grand (qwen3:8b)	≈ 3	≈ 270	≈ 3 s	≈ 13,5 min
Moyen (qwen3:4b)	≈ 8	≈ 715	≈ 1,5 s	≈ 18 min
Petit (qwen3:1.7b)	≈ 13	≈ 1 170	≈ 0,8 s	≈ 15,6 min
Total	**≈ 24**	**≈ 2 155**		**≈ 47 min + ≈ 6 min de chargements**

Avec des requêtes parallèles (deux fois plus rapide, hypothèse), un run tiendrait entre 25 et 55 minutes : dix runs représentent une demi-journée à une journée de calcul. La configuration « tout grand modèle » (E5) coûterait environ le double ou le triple et peut être exécutée à échelle réduite, à justifier comme le permet E2.

8. Où vit chaque donnée (F3) et schémas d'événements (F4)

Donnée	Besoin d'accès	Stockage	Justification
Flux d'événements	Dizaines de milliers par run, écriture continue, quelques ms, rétention courte (7 jours)	Kafka	Ordre garanti, découplage, relecture du flux.
État courant du monde	120 agents, lectures et écritures très fréquentes, latence < 5 ms, éphémère	Redis	Clé-valeur en mémoire, lu par agents et pont.
Historique des runs	Volume cumulé, écriture par lots, requêtes analytiques, conservation longue	Parquet + DuckDB	Colonnaire, SQL reproductible (F5), fichiers portables.
Base de connaissances	Quelques centaines de documents + milliers de mémoires, recherche par similarité en 10 à 50 ms	ChromaDB	Filtres sur agent, rôle et date simulée.
Journal de rejeu	Un fichier par run, lecture séquentielle	JSONL / Parquet	Rejoue le run sans appel LLM (E3).
Secrets et configuration	Quelques valeurs, lecture au démarrage	Fichier .env, YAML	Jamais versionnés.

Enveloppe commune à tous les événements. `event_id`, `schema_version`, `run_id`, `type`, `sim_time_occurred`, `sim_time_processed`, `wall_time_emitted`, `source_agent_id`, `zone`, `payload`, `caused_by` (identifiants des événements à l'origine, pour remonter des indicateurs aux événements bruts, F6). Un message qui ne respecte pas le schéma est routé vers `sim.dlq` sans arrêter la simulation ; les sorties invalides des agents alimentent `sim.incidents` (A4).

9. Base de connaissances et mémoire (F1, F2, A1, C6)

- **Collection « corpus_monde ».** Documents fictifs écrits par vous : communiqués de la marque, fiches produit (ingrédient et indice de protection **fictifs**), avis clients, articles de presse beauté, conseils de pharmacien, historique de lancements précédents. Métadonnées : type, date simulée de publication, zone, rôle de l'auteur.
- **Collection « mémoires ».** Épisodes vécus par chaque agent LLM (ce qu'il a vu, dit ou entendu) avec date simulée et importance. Un résumé périodique compresse les épisodes anciens, pour qu'un agent se souvienne d'un échange vieux de plusieurs jours simulés (A1).

- **Récupération.** Top-4 par similarité, filtré par `date_sim <= sim_time` de l'agent (aucun document du futur, C6) et par visibilité du rôle (un vendeur ne lit pas le courrier interne de la marque).
- **Évaluation (F2).** Échantillon d'une trentaine de décisions : journaliser les passages retrouvés, annoter leur pertinence (0 ou 1) et reporter la précision au rang 4. Le journal de récupération fait partie de chaque run.

10. Godot, interaction et rejeu

- **Affichage (D1).** Une scène à sept zones ; chaque agent est un marqueur dont la couleur indique le sens (quatre catégories) et le contour le palier (grand, moyen, petit, règles). Un panneau latéral affiche les indicateurs, synchronisé avec la scène.
- **Inspection (D2).** Un clic sur un agent interroge le pont : mémoire récente, justification de la dernière décision ou, pour un agent à règles, son état et la règle appliquée.
- **Injection (D3).** Un bouton publie un événement sur `sim.control` (rupture de stock, polémique virale, vague de chaleur) ; ses effets se propagent par le bus.
- **Rejeu (E3).** Le même pont lit un journal enregistré au lieu du flux Kafka et réémet les décisions déjà produites : aucune requête LLM, même visualisation, mêmes indicateurs. C'est aussi le filet de sécurité si un quota d'API tombe pendant la démonstration.

11. Plan d'expériences (E1, E2, E4, E5)

Configuration	Agents	Rôle dans la démonstration	Runs
C0 : règles seules	Les 120 agents à règles, mêmes scénarios et mêmes graines	Baseline de E1 : ce que le LLM apporte ou n'apporte pas. Quasi instantané, donc beaucoup de graines possibles.	≥ 5 (idéalement 20)
C1 : mixte (référence)	84 LLM sur 3 paliers + 36 règles	Configuration principale, mesure de l'émergence du sens (B2, B3).	≥ 5
C2 : tout grand modèle	Tous les agents LLM sur qwen3:8b	Référence de E5 : même apport pour combien d'appels de plus ? Échelle réduite possible, à justifier.	≥ 5
C3 : tout petit modèle (option)	Tous les agents LLM sur qwen3:1.7b	Borne basse du coût, isole l'effet de la capacité.	≥ 5

Coût (E4). Chaque appel journalise le modèle, les tokens en entrée et en sortie (fournis par Ollama dans sa réponse) et la durée ; chaque run totalise appels, tokens et temps. L'énergie peut s'estimer en échantillonnant la puissance du GPU avec `nvidia-smi` ou avec un outil de mesure dédié, à valider. **Piège à traiter (E1).** Le déclin de fin d'été peut s'expliquer par la seule saisonnalité : la baseline à règles doit avoir la même saisonnalité, et il faut montrer un déclin d'usage en cours de saison.

12. Prérequis

Catégorie	Élément	Remarque
Système	Windows 11 à jour avec WSL2 (à confirmer : le système n'est pas précisé)	Nécessaire pour Docker Desktop.
GPU	Pilote NVIDIA récent	Vérifier avec `nvidia-smi` : VRAM totale et version du pilote.
Langage	Python 3.11 ou 3.12, environnement virtuel	Orchestrateur, pont, indicateurs, scripts de mesure.
Conteneurs	Docker Desktop (moteur WSL2)	Pour Kafka et Redis uniquement. Ollama reste sur l'hôte.
Inférence	Ollama pour Windows	Gère le GPU directement, plus simple qu'un passage GPU dans Docker.
Visualisation	Godot 4.x, version stable	Scène, UI, client WebSocket.
Versionnage	Git et un dépôt privé partagé avec la collaboratrice	Les clés API ne doivent jamais y figurer.
Espace disque	Environ 10 Go de modèles, 3 Go d'images Docker, quelques Go de runs	Le SSD de 1 To suffit largement.
Comptes	GitHub, et facultativement Groq, Google AI Studio, GitHub Models	Voir section 14.
Binôme	Machine de la collaboratrice : GPU, VRAM, système	Le livrable doit tourner sur une autre machine : documenter au moins la configuration réduite.

13. Initialisation pas à pas

Les commandes sont à lancer dans un terminal PowerShell. Les versions précises (Python, Kafka, Godot) sont à figer dans le dépôt au premier jour.

#	Étape	Commande ou action	Vérification
0	**Contrôler la carte graphique**	`nvidia-smi`	La VRAM totale affichée est d'environ 8 Go.
1	**Installer Ollama**	Télécharger l'installeur sur ollama.com/download, puis `ollama --version`	La commande répond.

2	**Télécharger les modèles**	`ollama pull qwen3:1.7b`
		`ollama pull qwen3:4b`
		`ollama pull qwen3:8b`
		`ollama pull nomic-embed-text`
			`ollama list` affiche les quatre modèles (à confirmer dans le catalogue au moment du téléchargement).

3	**Tester un modèle**	`ollama run qwen3:1.7b` puis une question courte ; ensuite `ollama ps`	La colonne PROCESSOR indique 100 % GPU.
4	**Régler Ollama**	Variables utilisateur Windows puis redémarrage d'Ollama : `OLLAMA_MAX_LOADED_MODELS=2`, `OLLAMA_NUM_PARALLEL=2` (monter à 4 après mesure), `OLLAMA_KEEP_ALIVE=30m`, `OLLAMA_FLASH_ATTENTION=1`	À vérifier dans la documentation de votre version.
5	**Limiter WSL2**	Fichier `%UserProfile%\.wslconfig` : `[wsl2] memory=8GB processors=8`, puis `wsl --shutdown`	Docker ne dépasse plus 8 Go.
6	**Lancer Kafka et Redis**	Un fichier `docker-compose.yml` avec l'image Apache Kafka (un broker KRaft, tas limité à 1 Go) et Redis, puis `docker compose up -d`	`docker ps` montre les deux conteneurs ; création des topics.
7	**Créer l'environnement Python**	`python -m venv .venv` puis activation. Paquets : ollama, aiokafka, pydantic, fastapi, uvicorn, chromadb, duckdb, redis, pandas, pyarrow, python-dotenv, pytest	`pip list` ; un script de test importe tout.
8	**Mesurer la machine (benchmark)**	Script qui envoie un prompt de 600 tokens et lit 80 tokens JSON, par modèle, en séquentiel puis en parallèle (2, 4, 6) ; résultats dans un tableau	Remplace les latences supposées de la section 7 ; sert aussi à E4.
9	**Structurer le dépôt**	Dossiers : sim, agents, llm, bus, rag, indicators, bridge, replay, corpus, godot, runs, docs. `.gitignore` : .env, .venv, runs/, données Docker	Premier commit sans secret.

#	Étape	Commande ou action	Vérification
10	**Première tranche verticale**	3 agents LLM (un par palier) et 1 agent à règles, 5 pas, événements publiés sur Kafka et lus par un client console	Le flux complet fonctionne, même minimal.
11	**Brancher Godot**	Pont FastAPI + scène minimale : un carré par agent qui change de couleur selon le sens	On voit les décisions en direct.
12	**Corpus et RAG**	Écrire 20 documents fictifs, les indexer dans ChromaDB, tester la récupération avec filtre de date	Les passages retrouvés sont pertinents et jamais du futur.
13	**Indicateurs et rejeu**	Entropie du sens par zone, écrivain d'historique, lecteur de rejeu	Un run enregistré se rejoue sans appel LLM.

14. Clés API : lesquelles, où les obtenir

> **Le cœur du projet n'a besoin d'aucune clé.** Ollama tourne en local. Les clés servent de renfort (un palier alternatif, des tests de comparaison) et de plan B. Les quotas gratuits changent souvent : **à vérifier au moment de l'inscription**. Les valeurs ci-dessous viennent des pages officielles consultées à la date de ce document.

Fournisseur	Création de la clé	Limites gratuites constatées	Points d'attention
Groq	console.groq.com, créer un compte, rubrique API Keys, « Create API Key »	Page de limites : pour gpt-oss-20b et gpt-oss-120b, 30 requêtes/min, 1 000 requêtes/jour, 8 000 tokens/min, 200 000 tokens/jour. À confirmer pour votre plan dans les paramètres du compte.	Bon candidat pour un palier « grand » alternatif : quelques centaines d'appels par run.
Google AI Studio (Gemini API)	aistudio.google.com, « Get API key » ; la clé est rattachée à un projet	Limites par projet : requêtes/min, tokens/min, requêtes/jour, remises à zéro à minuit heure du Pacifique ; valeurs par modèle dans AI Studio. Modèles préliminaires plus restreints.	Les limites dépendent du modèle : relever les chiffres à l'inscription.
GitHub Models	GitHub, Settings, Developer settings, jeton d'accès personnel avec la permission models:read	Palier Copilot Free : 15 requêtes/min et 150/jour (modèles « low »), 10 et 50 (modèles « high ») ; 8 000 tokens en entrée et 4 000 en sortie par requête.	Limites susceptibles de changer sans préavis. Quotas faibles : réservé aux tests.
OpenRouter	openrouter.ai, rubrique Keys	Modèles « :free » : 20 requêtes/min et 50 requêtes/jour ; 1 000/jour seulement après un achat d'au moins 10 crédits.	Non retenu : 50 par jour est trop bas, et le palier supérieur n'est plus gratuit.
Mistral (plan Experiment)	Inscription à La Plateforme, plan gratuit Experiment ; numéro de téléphone vérifié, une seule utilisation par numéro, sans carte bancaire	Limites non précisées sur la page consultée.	Les requêtes du plan Experiment peuvent servir à entraîner les modèles de Mistral : n'envoyer que des données fictives.

Recommandation. Créez dès maintenant les clés **Groq** et **Google AI Studio**, puis GitHub Models si besoin. Chaque membre du binôme crée sa propre clé : les limites sont rattachées au compte ou au projet, et il ne faut pas multiplier les comptes pour contourner les quotas. **Sécurité.**

- Stocker les clés dans un fichier `.env` non versionné, avec un fichier `.env.example` sans valeurs commité dans le dépôt.
- Charger les clés au démarrage (python-dotenv) et ne jamais les écrire dans un prompt, un notebook, une capture d'écran ou un message de commit.
- En cas de fuite : révoquer la clé immédiatement dans la console du fournisseur et en créer une nouvelle.
- N'envoyer aux offres gratuites que des données fictives du monde : certaines réutilisent les requêtes.

15. Alternatives écartées

Alternative	Pourquoi elle est écartée
Tous les agents sur des API gratuites	Quotas de l'ordre de 1 000 requêtes par jour contre environ 2 000 appels par run, plus dix runs minimum : insuffisant. Latence variable et données partagées.
Un seul grand modèle pour tous les agents	Contraire à la sobriété demandée (E5) et trop lent sur 8 Go de VRAM ; sert uniquement de référence (C2).
Modèles de familles différentes selon les profils	Ajoute un facteur de variation non isolé et brouille l'interprétation de A5.
Framework multi-agents (LangGraph, CrewAI, AG2, etc.)	Facultatifs dans l'énoncé ; ils masquent l'horloge et les appels, ce qui complique la justification de C3, C4 et C6.
PostgreSQL pour tout	Plus lourd à exploiter ; Parquet et DuckDB suffisent pour une analytique mono-utilisateur et se transportent comme de simples fichiers.
Godot relié directement à Kafka	À ma connaissance, pas de client Kafka natif pour Godot ; un pont WebSocket est plus simple et sert aussi l'inspection et l'injection.
Ollama dans Docker	Le passage du GPU sous Windows ajoute de la complexité sans bénéfice ; Ollama natif est plus simple et plus rapide à diagnostiquer.

16. Risques et parades

Risque	Effet	Parade
VRAM saturée ou rechargements fréquents	Latence imprévisible, runs lents	Ordonnancement par palier, `ollama ps` pour surveiller, contexte court, parallélisme progressif.
Petits modèles qui sortent du rôle ou cassent le JSON	Décisions inexploitables	Schéma imposé, validation, une relance, repli sur règle, incidents comptés (A4).
Bridage thermique du portable	Mesures de temps faussées (E4)	Secteur, profil de performance, support ventilé, pauses entre runs, températures consignées.
Connaissances du LLM sur les crèmes solaires réelles	Résultats attribuables au modèle plutôt qu'au monde	Marque, ingrédient et indice fictifs, catégories fermées, test de contamination du nom.
Incohérence temporelle	Agent qui agit sur une information future	Délai de livraison, filtre de date dans le RAG, violations comptées (C6).
Quota d'API tombé pendant la démonstration	Démonstration bloquée	Cœur 100 % local et rejeu (E3).
Saisonnalité qui explique tout	Aucun apport démontré du LLM	Même saisonnalité dans la baseline à règles, déclin mesuré en cours de saison.
Allégations de santé dans le corpus	Contenu problématique	Produit strictement cosmétique, aucune promesse médicale, indices de protection fictifs.

17. Décisions à valider

- La répartition des 120 agents et les proportions de paliers de la section 5.
- Les trois paliers Qwen3 (1,7 b, 4 b, 8 b) et le palier grand en local plutôt que par API.
- La durée simulée (90 jours, un pas par jour) et les quatre catégories de sens (efficace, tendance, banal, dépassé).
- Le choix Redis, Parquet + DuckDB et ChromaDB, et l'absence de framework multi-agents.
- Le système d'exploitation et la configuration de la machine de la collaboratrice.
- Le nom de la marque, à choisir avec un test de contamination avant les premiers runs.

Sources consultées (liens cliquables) : Small language models 2026 · Fastest local LLMs (low-end PCs) · Gemini API : limites · Groq : limites · OpenRouter : limites · GitHub Models · Mistral : plan Experiment

Les tailles de modèles, vitesses et quotas proviennent de sources tierces ou de pages susceptibles de changer ; les latences de la section 7 sont des hypothèses de dimensionnement, à remplacer par les mesures de votre machine.